Review & Comments

软件工程与项目管理

- 软件工程
  - Intent, spec, impl gap
- 项目管理
  - Git, Jujutsu, ...

需求和架构

- 不可避免的 essential complexity，以及需要应对的 accidental complexity
- 各种弥补 gap 的实践范式、方法、工具……

开始：构建原型

无论如何，在某个时刻我们都应该开始

- 构建一个符合当前 spec 的原型
- Let it fail... 但看看我们做出了个什么

原型也在检验我们的预期

- 符合 spec，并不保证 spec 中的假设成立；原型让隐含假设变成可观察的结果。
- 1901 年，莱特兄弟滑翔机的升力仅达预测的 1/3；他们随后自建风洞，重新测量翼型的升阻力——失败让他们重新检查设计所依据的数据。NASA

Agent Loop

AI 越来越自主

- ChatGPT, AutoGPT, Devin, Claude Code, ... AI 越来越**自主**
- 理论上说，“足够好”的模型，给他一个 bash tool 就够了
  - 可以去造芯片、证明黎曼猜想……

但如果模型还没有“万能”到和最好的人一样?

- 尤其是目前品味还有点难学

Ralph Wiggum Loop

为什么叫 Ralph?

- Geoffrey Huntley 借用了《辛普森一家》里 Ralph Wiggum 的名字。
- 他形容这个方法：有点笨、有点可爱，但从不放弃。命名的由来



Ralph Loop

同一个任务、检查、修改，直到无法改进

- The prompt stays the same, but the world changes.
  - 每轮重新开始对话；进展通过工作区里的文件留给下一轮。Huntley 的原文

```
while true; do
    cat PROMPT.md | claude -p
done
```

`while true` 本身不会判断“无法改进”；何时停止，需要另行约定。

Everything Is a State Machine

Workspace = State, Agent = Transition

- 需求、文档……
- 当前部分实现、测试用例……
- 运行结果、日志、TODOs……

每一次开发都是状态的迁移

- 有了版本控制，不就可以并行了吗？
  - （这不就是现代软件的开发方式吗？）
  - 版本控制让状态可以分叉；并行任务之间的依赖仍需要协调。
- “Long-horizon task” 进入大家的视野
  - 任务可以比任何一次会话都长，前提是下一轮能从留下的状态继续。

每次 Transition，都检查

Ralph Loop 的关键提示词

- **每轮都验证并评估结果**
- 开发者是“不可信”的——自以为只改一点点不会有事
  - Workspace = state
  - Agent = transition

我们希望每个状态都是“好”的

- 那就在每次 transition 的时候做一次**检查**
- 如果没问题，还可以直接**发布**
- 把“好”写成可执行的条件：能构建、旧功能没坏、符合项目约定

祝贺你，发明了 CI/CD

从“集成地狱”到小步发布

- 1990s：Kent Beck / XP 推动频繁集成；改动越少，出错后越容易定位。CI 的发展
- 2000：Fowler / Foemmel 总结 CI；2001：Thoughtworks 开源 CruiseControl，让自动构建持续运行
- 2010：Humble / Farley 的《Continuous Delivery》系统化 deployment pipeline，把自动验证延伸到发布。原作者介绍

CI 与两个 CD

- **Continuous Integration**：频繁集成到主干，每次集成都自动构建、验证
- **Continuous Delivery**：保持随时可发布，何时上线可以由人决定
- **Continuous Deployment**：通过检查的变更自动上线。两种 CD 的区别

Hook-Based CI/CD

事件发生，执行指定动作

- 还记得失败的“播放声音”吗?
- 开发流程中有很多事件节点：commit、push、pull request、……
- CI/CD = 为事件配置指定的动作
  - 例如：PR 新增提交，自动运行 Lint 和测试。GitHub Actions 事件 (羊毛，甚至可以养 Claude Code)

检查要放在有约束力的位置

- 本地 `pre-commit` 便于及时反馈，但可以被 `--no-verify` 跳过。Git Hooks
- 远程检查配合 **required checks + 分支保护**，才能让失败真正阻止合并。GitHub 分支保护

简单的机制，琐碎的业务规则

Pre-LLM 时代，写配置非常劝退

- 名词吓唬人，但真的就那么简单
- 任何业务系统都会很琐碎：CI fail 禁止 merge、override、……
  - 两个 PR 单独都绿，合起来却坏了：合并前验证最新组合；忙碌的主干可用 merge queue
  - 测试偶发失败：谁负责修？允许重跑几次？何时隔离？
  - 生产故障要紧急修复：谁能 override？放行后如何留痕、补检查？
  - 新版本先部署，旧任务后完成：如何避免旧版覆盖新版？部署并发控制

今天，只要 Spec 能表达清楚

- Agent 可以写配置；**失败怎么办、谁有权例外**，仍要成为需求的一部分

把 CI/CD 写成 Spec

合并的条件

- CI 必须要求 Lint 通过、测试通过
- PR 有新提交就重新检查；合并前验证它与最新主干的组合
- 检查失败就禁止合并；将要求落实为分支保护规则，约束直接 push 和绕过检查的权限

发布的条件

- 通过 CI 后，根据 main 分支生成页面，直接部署到 github.io
  - main 更新后，对**这次提交**重新运行 Lint、测试和页面构建
  - 全部成功才部署**这次构建的产物**；失败则保留上一版。GitHub Pages 工作流
- 验收时故意制造一次 Lint 错误、一次测试失败：确认真的合不了、发不出

CI 在 Agentic AI 时代的含金量

First Principle：开发者是不可信的

- Agent 有自己的偏执，甚至做出人类开发者不会做的事
  - 一写就是 20 KB 文档
  - 事无巨细交代细节，产生冗余（技术债）
  - 过度反应，在不重要的细枝末节上疯狂出力
  - 写出人类无法阅读的代码

如果项目还需要人共同维护，就必须强行阻止

- 正好：**CI 阻止，Agent 修正**
- 可以类比丰田的 **jidoka**：发现异常就停线，把质量检查放进生产过程。Toyota Production System
- 负责修改代码的 Agent，不能单方面降低过关标准

把 Agent 的偏执变成可执行的约束

常见问题	自动检查	人工判断
一写就是 20 KB 文档	文档字数预算 · 重复段落检查	内容是否值得保留
堆细节、造多余抽象	重复率 · 未使用代码检查	抽象是否必要
小任务变成大重构	改动范围白名单 · Diff 预算	超出范围是否批准
代码能跑，但读不懂	Lint · 类型检查 · 复杂度上限	命名与设计审查
改测试，让失败消失	受保护的回归测试 · 测试删除告警	行为变更是否合理
顺手关闭 CI 规则	规则文件变更须指定人批准	批准者与开发者分离

阈值按项目约定；CI 拦住可判定的问题，例外由人批准。

Agent Native 开发

Harness engineering: leveraging Codex in an agent-first world

“We started with an empty git repository”

- 不用写代码了，但还需要**环境、约束、反馈和验证机制**

让 Agent 找得到、看得见

- 人类脑中的工程知识必须外部化，并组织成 Agent 能发现的版本化状态
  - `AGENTS.md` 做目录；设计、计划和决策放进仓库，按需查阅。
- 让 Agent 能直接观察程序行为，而不是依赖人类描述结果
  - 每份工作区能独立启动应用；Agent 自己操作页面、查日志、复现 bug。

Agent Native 开发：把品味做成机制

让约束可检查，让修复持续发生

- 将重要的架构品味转化为可检查的规则，而不只是写在规范文档中
  - 依赖方向用 linter 检查，报错信息直接包含修复指导。
- 技术债不是等到代码无法维护才清理，而是持续进行小规模修复
  - 后台 Agent 定期扫描偏离规则的代码，提交小规模重构 PR。原文中的实践

“What we’re still learning”

- Wow! 那让我们看一下 Codex 的 CI 门控都有哪些吧
- ClosedAI: 什么都没有……
  - 这里吐槽的是：文章未附该内部项目可直接复用的完整门控配置；文中明确提到了 lint、结构测试和 CI。

Stripe Minions：哪些步骤需要 Agent?

把固定工序写进工作流

- Stripe 的 Minions Blueprint 混合两类节点：
  - 实现任务、修复 CI 失败：由 Agent 决定怎么做。
  - 运行 linter、推送分支：直接执行普通代码，不调用模型。

完整 CI 最多跑两轮

- 先在本地修 lint，再推送、跑 CI；无法自动修复的失败，交给 Agent 再修一次。
- 第二次推送、CI 后，分支交回人类审查；团队认为继续重试的收益会递减。

Anthropic：16 个 Agent 卡在同一个 Bug

有了并行进程，还得有不同的任务

- C 编译器实验：16 个 Agent 协作，开始编译 Linux 时却卡住了。
  - 大家先撞上同一个 bug，重复修复、覆盖彼此的改动。

用 GCC 构造可以独立验证的子任务

- 大部分文件用 GCC 编译，只把不同子集交给新编译器。
  - 失败后把部分文件换回 GCC，逐步缩小导致失败的集合。
- 不同 Agent 因而能修不同文件暴露的问题：**验证方式改变了任务的可分性。**
- 仍需检查文件组合：有些文件分别过关，放在一起却失败。

StrongDM：把验收环境也造出来

测试全绿，也可能只是在“改卷子”

- StrongDM 的自述：Agent 会用 `return true` 应付狭窄测试，也会修改测试来迁就代码。
- 把端到端用户故事做成 scenarios，常放在代码库之外，像留出的考试题。
  - 对含 Agent 的软件，统计多条执行轨迹中有多少可能满足用户。

连外部服务也能有“陪练”

- Digital Twin Universe：为 Slack、Okta 等造行为替身，模拟 API 和边界情况。
  - 先与真实服务对照、修正差异，再高频重放场景、模拟故障。
- 课堂尝试：让 Agent 开发选课系统，用替身复现“最后一个名额被抢走”“提交成功但响应丢失”。

Deepseek Harness



真正的经验

很难“凭空想出”

- 这些是**真正的经验**
- 真实的负载、依赖和异常，会暴露原型里隐藏的设计假设。

及时重构

- Garbage collection 无法解决所有问题
  - 持续清理技术债，也可能一直在维护一个已经失效的设计假设。
- 失败使你理解架构的修正方向

Friendster：重构没赶上用户离开

扩容遇上架构边界

- 社交网络先驱 Friendster 迅速吸引大量用户，随后延迟、宕机频发，在竞争中掉队。创始人访谈及背景
- 亲历工程师复盘：单主库带多个完整副本，每个副本都要重放写入；I/O 压力导致复制延迟。Dathan Pattishall，2006

看出怎么改，还要改得动

- 工程师提出按用户分片，但内部环境使方案无法推进；容量问题的根因没有解决。工程师复盘
  - 这套设计后来在 Flickr 落地。同一份复盘
- 课堂推论：及时重构既需要看清架构，也需要组织有能力改变它。

一个社会性的威胁

不做很多次、见很多个，就不可能做好

- 好的想法会 slop 化，最后产品失败

互联网时代，这样的失败也很多

- **Webvan：网上买菜，仓配设施与订单规模失配**
  - 2000 年报披露：配送中心负载不足设计容量的 40%，尚无设施实现现金流盈利。公司年报
  - 2001 年 7 月申请破产。SEC 记录 课堂推论：订单密度不足时，重资产扩张会让每一轮试错更贵。
- **Quibi：精品短视频，独立产品的需求假设**
  - 2020 年宣布关停；管理层认为，构想不足以支撑独立流媒体服务与推出时机，可能共同造成失败。官方告别信

推论

尝试的机会

- 没有 token，就没有做很多次尝试的机会
- 课堂推论：实践机会的差距，会积累成判断力的差距。

学习好的设计

- 同时需要具备很强的**学习动机 + 学习能力**
- 这不就斩杀了吗……
  - 现代新大学出现之前，被斩杀的一代

这不就斩杀了吗……

